SMART CONTRACT AUDIT REPORT

for

Melt Finance

Prepared By: Xuxian Jiang

PeckShield

April 13, 2026

Document Properties

Client	Melt Finance
Title	Smart Contract Audit Report
Target	Melt Finance
Version	1.0.1
Author	Xuxian Jiang
Auditors	Matthew Jiang, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0.1	April 13, 2026	Xuxian Jiang	Post-Final Release #1
1.0	March 26, 2026	Xuxian Jiang	Final Release
1.0-rc	March 25, 2026	Xuxian Jiang	Release Candidate

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xuxian Jiang
Email	contact@peckshield.com

Contents

1 Introduction 4
- 1.1 About Melt Finance . . . 4
- 1.2 About PeckShield . . . 5
- 1.3 Methodology . . . 5
- 1.4 Disclaimer . . . 7

2 Findings 9
- 2.1 Summary . . . 9
- 2.2 Key Findings . . . 10

3 Detailed Results 11
- 3.1 Revisited Owner Assignment in MeltHub . . . 11
- 3.2 Timely Minter Role Revocation After HyperCore Integration Preparation . . . 13
- 3.3 Improved Token Admin Transfer Logic in MeltFactory . . . 14
- 3.4 Trust Issue of Admin Keys . . . 15

4 Conclusion 17

References 18

1 Introduction

Given the opportunity to review the design document and related smart contract source code of the `Melt Finance` protocol, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About Melt Finance

`Melt Finance` is the native spot infrastructure built for the `Hyperliquid` ecosystem. Acting as the foundational layer for on-chain assets, `Melt` directly lists tokenized `Real-World Assets` (equities, commodities, `ETFs`) and crypto assets as native `HIP-1` spot tokens. Powered by an integrated bridge and swap engine, users can acquire these `mAssets` in a single click from any chain, delivering a truly frictionless spot trading experience. The basic information of the audited protocol is as follows:

Table 1.1: Basic Information of The `Melt Finance` Protocol

Item	Description
Issuer	Melt Finance
Type	Ethereum Smart Contract
Platform	Solidity
Audit Method	Whitebox
Latest Audit Report	April 13, 2026

In the following, we show the Git repository of reviewed files and the commit hash value used in this audit.

- https://github.com/Melt-Protocol/contracts.git (1728acb)

And this is the commit ID after all fixes for the issues found in the audit have been checked in:

- https://github.com/Melt-Protocol/contracts.git (1194658, b2994a4)

1.2 About PeckShield

PeckShield Inc. [9] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (https://t.me/peckshield), Twitter (http://twitter.com/peckshield), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification

Impact	Likelihood: *High*	Likelihood: *Medium*	Likelihood: *Low*
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [8]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

contract is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [7], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logics	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 | Findings

2.1 Summary

Here is a summary of our findings after analyzing the `Melt Finance` implementation. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	1	■
Low	3	■ ■ ■
Informational	0	
Total	4	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities that need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in Section 3.

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 1 medium-severity vulnerability and 3 low-severity vulnerabilities.

Table 2.1: Key Melt Finance Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Low	Revisited Owner Assignment in MeltHub	Business Logic	Resolved
PVE-002	Low	Timely Minter Role Revocation After HyperCore Integration Preparation	Business Logic	Resolved
PVE-003	Low	Improved Token Admin Transfer Logic in MeltFactory	Coding Practices	Resolved
PVE-004	Medium	Trust Issue of Admin Keys	Security Features	Mitigated

Besides recommending specific countermeasures to mitigate these issues, we also emphasize that it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms need to kick in at the very moment when the contracts are being deployed in mainnet. Please refer to Section 3 for details.

PeckShield

3 Detailed Results

3.1 Revisited Owner Assignment in MeltHub

- ID: PVE-001
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: MeltHub
- Category: Business Logic [6]
- CWE subcategory: CWE-841 [3]

Description

The Melt Finance protocol has a central hub on HyperEVM for multi-token bridging and wrapping. This central hub named MeltHub is designed for cross-chain bridge, same-chain wrap/unwrap, and HyperCore integration. While reviewing the role assignment in MeltHub, we notice a need to revisit current role assignment.

To elaborate, we show below the implementation of two example routines, i.e., registerToken() and setFee(). The former is to register a new wrapper token while the latter configures the protocol-wide fee parameter. We notice both routines have the onlyOwner modifier, indicating they are managed by the same account. However, our analysis shows that the first routine is designed to be called by the MeltFactory contract. But the second routine is indeed managed by a multi-sig account. With that, current admin account needs to be revisited with a role-based access control mechanism. Note it affects a number of admin-related privileged functions in MeltHub.

```
    function registerToken(address originalToken_, address wrappedToken_, uint32
        sourceEid_, uint256 feeBps_)
        external
        onlyOwner
    {
        if (originalToken_ == address(0)  wrappedToken_ == address(0)) {
            revert MeltErrors.InvalidAddress();
        }
        if (feeBps_ > MeltTypes.MAX_FEE_BPS) revert MeltErrors.InvalidFee();

        bytes32 configKey = _getConfigKey(originalToken_, sourceEid_);
```

```
        if (tokenConfigs[configKey].isRegistered) revert MeltErrors.TokenAlreadyRegistered();

        tokenConfigs[configKey] = MeltTypes.TokenConfig({
            wrappedToken: MeltAsset(wrappedToken_),
            originalToken: originalToken_,
            sourceEid: sourceEid_,
            feeBps: feeBps_,
            paused: false,
            isRegistered: true
        });
        wrappedToConfigKey[wrappedToken_] = configKey;

        emit MeltEvents.TokenRegistered(configKey, originalToken_, wrappedToken_, sourceEid_);
    }

    function setFee(bytes32 configKey, uint256 feeBps_) external onlyOwner {
        if (!tokenConfigs[configKey].isRegistered) revert MeltErrors.TokenNotRegistered();
        if (feeBps_ > MeltTypes.MAX_FEE_BPS) revert MeltErrors.InvalidFee();

        uint256 oldFee = tokenConfigs[configKey].feeBps;
        tokenConfigs[configKey].feeBps = feeBps_;

        emit MeltEvents.FeeUpdated(configKey, oldFee, feeBps_);
    }
```

Listing 3.1: `MeltHub::registerToken()/setFee()`

Recommendation Revisit the above admin account with a new role-based access control mechanism.

Status The issue has been fixed in the following commit: `76229f5`.

3.2 Timely Minter Role Revocation After HyperCore Integration Preparation

- ID: PVE-002
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `MeltFactory`
- Category: Business Logic [6]
- CWE subcategory: CWE-841 [3]

Description

`Melt Finance` is in essence a cross-chain token bridge using `LayerZero V2` with `HyperEVM/HyperCore` integration. For each supported token, it creates a new wrapper token and assigns the roles of `MINTER_ROLE` and `BURNER_ROLE` to the `MeltHub` contract. In the meantime, it also grants the factory itself the `MINTER_ROLE` for `HyperCore` integration. Our analysis shows that the `MINTER_ROLE` role for itself can be revoked once the integration is completed.

To elaborate, we show below the implementation of the related `prepareForCore()` routine. As the name indicates, it is designed to prepare the given token for `HyperCore` integration. As part of the integration, it exercises the `MINTER_ROLE` role to mint the specified `mintAmount` to the associated `assetBridge`. Note it is executed only once and the `MINTER_ROLE` role can be immediately revoked after the execution.

```
    function prepareForCore(bytes32 configKey, uint64 expectedCoreIndex_, int8
        decimalDiff_, uint256 mintAmount)
        external
        onlyOwner
    {
        PendingDeployment storage d = deployments[configKey];
        if (address(d.wrappedToken) == address(0)) revert MeltErrors.DeploymentNotFound
            ();
        if (!d.registered) revert MeltErrors.NotRegistered();
        if (d.corePending) revert MeltErrors.AlreadyPrepared();

        address assetBridge = HyperCoreLib.getAssetBridge(expectedCoreIndex_);
        d.wrappedToken.mint(assetBridge, mintAmount);

        d.expectedCoreIndex = expectedCoreIndex_;
        d.decimalDiff = decimalDiff_;
        d.corePending = true;

        emit MeltEvents.CorePrepared(configKey, address(d.wrappedToken),
            expectedCoreIndex_, assetBridge, mintAmount);
    }
```

Listing 3.2: `MeltFactory::prepareForCore()`

Recommendation Revoke the `MINTER_ROLE` role from the factory contract once the `HyperCore` integration is prepared.

Status The issue has been fixed in the following commit: `76229f5`.

3.3 Improved Token Admin Transfer Logic in MeltFactory

- ID: PVE-003
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `MeltFactory`
- Category: Coding Practices [5]
- CWE subcategory: CWE-1126 [1]

Description

The `Melt Finance` protocol has a core factory for deploying wrapped tokens and managing `HyperCore` integration. While reviewing a specific function to transfer the admin role of a wrapped token, we notice the current implementation may be improved.

To elaborate, we show below the implementation of the related `transferTokenAdmin()` routine. It has a rather straightforward logic in transferring the admin role of a wrapped token to a new address. It comes to our attention that this function does not validate the given new admin address should not be equal to itself. Otherwise, it may permanently lose the admin role over the related wrapped token. To fix, we need to ensure the given new admin address does not equal the factory contract itself.

```
    function transferTokenAdmin(bytes32 configKey, address newAdmin) external onlyOwner
        {
        PendingDeployment storage d = deployments[configKey];
        if (address(d.wrappedToken) == address(0)) revert MeltErrors.DeploymentNotFound
            ();

        MeltAsset token = d.wrappedToken;
        token.grantRole(token.DEFAULT_ADMIN_ROLE(), newAdmin);
        token.revokeRole(token.DEFAULT_ADMIN_ROLE(), address(this));

        emit MeltEvents.TokenAdminTransferred(configKey, address(token), newAdmin);
    }
```

Listing 3.3: `MeltFactory::transferTokenAdmin()`

Recommendation Revise the above routine to ensure the given new admin address is not the factory itself.

Status The issue has been fixed in the following commit: `b628c79`.

3.4 Trust Issue of Admin Keys

- ID: PVE-004
- Severity: Medium
- Likelihood: Medium
- Impact: Medium
- Target: `Multiple Contracts`
- Category: Security Features [4]
- CWE subcategory: CWE-287 [2]

Description

In the audited protocol, there is a privileged account `owner` that plays a critical role in governing and regulating the system-wide operations (e.g., configure the fee-related parameters, whitelist tokens to bridge, and pause/unpause the bridge). Our analysis shows that the privileged account needs to be scrutinized. In the following, we examine the privileged account and the related privileged accesses in current contracts.

```
    function setFee(bytes32 configKey, uint256 feeBps_) external onlyOwner {
        if (!tokenConfigs[configKey].isRegistered) revert MeltErrors.TokenNotRegistered
            ();
        if (feeBps_ > MeltTypes.MAX_FEE_BPS) revert MeltErrors.InvalidFee();

        uint256 oldFee = tokenConfigs[configKey].feeBps;
        tokenConfigs[configKey].feeBps = feeBps_;

        emit MeltEvents.FeeUpdated(configKey, oldFee, feeBps_);
    }

    function pauseToken(bytes32 configKey) external onlyOwner {
        if (!tokenConfigs[configKey].isRegistered) revert MeltErrors.TokenNotRegistered
            ();
        tokenConfigs[configKey].paused = true;
        emit MeltEvents.TokenPausedEvent(configKey);
    }

    function unpauseToken(bytes32 configKey) external onlyOwner {
        if (!tokenConfigs[configKey].isRegistered) revert MeltErrors.TokenNotRegistered
            ();
        tokenConfigs[configKey].paused = false;
        emit MeltEvents.TokenUnpausedEvent(configKey);
    }

    function pause() external onlyOwner {
        _pause();
    }

    function unpause() external onlyOwner {
        _unpause();
    }
```

```

    function setFeeCollector(address newCollector) external onlyOwner {
        if (newCollector == address(0)) revert MeltErrors.InvalidAddress();
        emit MeltEvents.FeeCollectorUpdated(feeCollector, newCollector);
        feeCollector = newCollector;
    }

    function setFeeController(address controller) external onlyOwner {
        emit MeltEvents.FeeControllerUpdated(address(feeController), controller);
        feeController = IMeltFeeController(controller);
    }

    function setCoreConfig(bytes32 configKey, uint64 coreIndexId_, int8 decimalDiff_)
        external onlyOwner {
        if (!tokenConfigs[configKey].isRegistered) revert MeltErrors.TokenNotRegistered
            ();
        coreConfigs[configKey] =
            MeltTypes.CoreConfig({coreIndexId: coreIndexId_, decimalDiff: decimalDiff_,
                isConfigured: true});
        emit MeltEvents.CoreConfigured(configKey, coreIndexId_, decimalDiff_);
    }
```

Listing 3.4: Example Privileged Functions in `MeltHub`

We understand the need of the privileged functions for contract maintenance, but at the same time the extra power to the owner may also be a counter-party risk to the protocol users. It would be worrisome if the privileged administrative account is a plain EOA account. Note that a multi-sig account could greatly alleviate this concern, though it is still far from perfect. Specifically, a better approach is to eliminate the administration key concern by transferring the role to a community-governed multisig or DAO. In the meantime, a timelock-based mechanism can also be considered as mitigation.

Recommendation Promptly transfer the privileged account to the intended `DAO`-like governance contract. All changed to privileged operations may need to be mediated with necessary timelocks. Eventually, activate the normal on-chain community-based governance life-cycle and ensure the intended trustless nature and high-quality distributed governance.

Status This issue has been mitigated as the team makes use of a multisig to act as the privileged owner.

4 Conclusion

In this audit, we have analyzed the design and implementation of `Melt Finance` protocol, which is the native spot infrastructure built for the `Hyperliquid` ecosystem. Acting as the foundational layer for on-chain assets, `Melt` directly lists tokenized `Real-World Assets` (equities, commodities, `ETFs`) and crypto assets as native `HIP-1` spot tokens. Powered by an integrated bridge and swap engine, users can acquire these `mAssets` in a single click from any chain, delivering a truly frictionless spot trading experience. During the audit, we notice that the current code base is well organized and those identified issues are promptly confirmed and fixed.

Meanwhile, we need to emphasize that smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

PeckShield

References

[1] MITRE. CWE-1126: Declaration of Variable with Unnecessarily Wide Scope. https://cwe.mitre.org/data/definitions/1126.html.

[2] MITRE. CWE-287: Improper Authentication. https://cwe.mitre.org/data/definitions/287.html.

[3] MITRE. CWE-841: Improper Enforcement of Behavioral Workflow. https://cwe.mitre.org/data/definitions/841.html.

[4] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[5] MITRE. CWE CATEGORY: Bad Coding Practices. https://cwe.mitre.org/data/definitions/1006.html.

[6] MITRE. CWE CATEGORY: Business Logic Errors. https://cwe.mitre.org/data/definitions/840.html.

[7] MITRE. CWE VIEW: Development Concepts. https://cwe.mitre.org/data/definitions/699.html.

[8] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

[9] PeckShield. PeckShield Inc. https://www.peckshield.com.